

FUNCTION RECURSION

It is an alternative process of iteration or loop where without execution of any loop statement we can get the desired output. Recursion means when a user defined function call itself within the body of itself.

To define any recursive function 4th type is required (function with return type & parameter passing)

Recursion

1. ~~Func~~ Function calls itself.
2. It is used (application) in function.
3. Smaller code size.
4. Used when code size needs to be small and time complexity is not an issue.
5. Time complexity is very high.
6. It can execute in a finite no of times.

Loop/Iteration

1. A set of instructions repeated executed.
2. It is used in case of loop.
3. Larger code size.
4. Used when time complexity needs to be balanced against and expended code size.
5. Relatively lower time complexity.
6. It can execute infinite no times.

- in case of recursion function 4th type is used only i.e with both return type & parameter passing is used.

1. WAP to print factorial of a no using recursion.

```
→ #include <stdio.h>
#include <conio.h>
int fact (int); // declaration
void main ()
{
    int n;
    printf ("Enter the no. = ");
    scanf ("%d", &n); // n=0
    printf ("The Factorial of the No. %d is %d", n, fact (n));
    getch ();
}
int fact (int n) // func. definition n=0
{
    if (n==0)
        return 1;
    else
        return n * fact (n-1); // 120 * fact (1-1);
}
```

2. WAP to print fibonacci series of a no. using recursion :-

- 1, 2, 3, 5, 8, 13, 21

```
#include <stdio.h>
#include <conio.h>
// int fibo (int); // func. declaration
void main ()
{
    int n, x;
    printf ("Enter the Range = ");
    scanf ("%d", &n); // n=5
    x = fibo (n);
    printf ("%d", x); // calling n=5 x=5
    getch ();
}
```

```

int fibo (int n) // func. definition n=1
{
    if (n==0)
        return 0;
    else if (n==1)
        return 1;
    else
        return fibo (n-1) + fibo (n-2); // calling
        recursion 5 + fibo (1-2)
}

```

3. Recursive C program to find sum of digits of a number :-

```

→ #include <stdio.h>
#include <conio.h>

// Function to check sum of digits using recursion
int sum-of-digit (int);
void main ()
{
    int num, result;
    printf ("Enter the no = ");
    scanf ("%d", & num); // num = 123
    result = sum-of-digit (num); // calling num = 123 result = 6
    printf ("Sum of digits in %d is %d\n", num, result);
    getch ();
}

int sum-of-digit (int n) // n=0
{
    if (n==0)
        return 0;
    else
        return (n%10 + sum-of-digit (n/10)); // 6 + sum-of-digit (0)
}

```


4. WAP to find out GCD and LCM of a number without recursion:-

```
→ #include <stdio.h>
#include <conio.h>
void main()
{
    int num1, num2, gcd, lcm, rem, nume, denom;
    printf("Enter two numbers\n");
    scanf("%d %d", &num1, &num2); // num1 = 27
                                     num2 = 3
                                     lcm = 27
                                     gcd = 3

    if (num1 > num2) // 27 > 3
    {
        nume = num1; // nume = 27
        denom = num2; // denom = 3
    }
    else
    {
        nume = num2;
        denom = num1;
    }

    rem = nume % denom; // 27 % 3 = 0
    while (rem != 0) // 0 != 0
    {
        nume = denom; // nume = 3
        denom = rem; // denom = 0
        rem = nume % denom; // rem = 3 % 0 = 0
    }

    gcd = denom; // gcd = 3
    lcm = num1 * num2 / gcd; // lcm = 81 / 3 = 27

    printf("GCD of %d and %d = %d\n", num1, num2, gcd);
    printf("LCM of %d and %d = %d\n", num1, num2, lcm);
    getch();
}
```

5. GCD of two integers :-

```
→ #include <stdio.h>
#include <conio.h>
int gcd (int, int);
int main()
{
    int n1, n2;
    printf ("Enter two positive integers = ");
    scanf ("%d %d", &n1, &n2); // n1=5 n2=0
    printf ("GCD of %d and %d is %d", n1, n2, gcd(n1, n2)); // calling
                                                                    n1=15 n2=20
    getch();
    return 0;
}

int gcd (int n1, int n2) // n1=5 n2=0
{
    if (n1 == 0) // 0!=0
        return gcd (n2, n1 % n2); // gcd(5,0) --> n2=5
    else
        return n1;
}
```

6. LCM of two integer without recursion :-

```
→ #include <stdio.h>
#include <conio.h>
int main()
{
    int n1, n2, min;
    printf ("Enter two positive integers : ");
    scanf ("%d %d", &n1, &n2); // n1=5 n2=3
    // maximum number between n1 and n2 is
    // stored in min
    min = (n1 > n2) ? n1 : n2; // min=5
    while (1)
    {
        if (min % n1 == 0 && min % n2 == 0)
        {
            printf ("The LCM of %d and %d is %d", n1, n2, min); // 15
            break;
        }
    }
```

```

    }
    ++min; // min = 15
}
getch();
return 0;
}

```

7. WAP a program power of a no. using recursion

```

→ #include <stdio.h>
#include <conio.h>
int power (int n1, int n2);
int main()
{
    int base, a, result;
    printf ("Enter base number: ");
    scanf ("%d", &base); // base = 3
    printf ("Enter power number (positive integer): ");
    scanf ("%d", &a); // a = 4
    result = power (base, a);
    printf ("%d ^ %d = %d", base, a, result);
    getch();
    return 0;
}

int power (int base, int a) // base = 3 a = 0
{
    if (a != 0)
        return (base * power (base, a-1)); // 81 * power (3, 0)
    else
        return 1;
}

```

8. WAP To calculate sum of the series using 4 type:-

```

→ #include <stdio.h>
#include <conio.h>
int soseries (int []); // func. declaration
int i, n;
void main()
{
    int a[5];
    printf ("Enter the Range: ");
}

```

```
scanf ("%d", &n); // n=5
// input
for (i=0; i<n; i++)
{
    printf ("Enter the no= ");
    scanf ("%d", &a[i]);
}
P = soseries(a); // calling
printf ("The Sum of the series is = %d", P);
getch();
}

int soseries (int x[]) // definition
{
    int s=0;
    for (i=0; i<n; i++)
    {
        s = s + x[i];
    }
    return s;
}
```